Types, Constants & Control Flow Cheatsheet

Lessons: 04 — Variables, Types & Constants · 05 — Control Flow

Examples: 04 · 05

Covers: every basic type, zero values, conversions, `iota`, and all four control-flow statements

Legend: `[*]` = real Go feature that the lessons have not covered yet

NUMERIC TYPES

```
int / uint                    platform-sized (64-bit on modern machines)
int8 int16 int32 int64        explicit signed widths
uint8 uint16 uint32 uint64    explicit unsigned widths
uintptr                   [*] an integer big enough to hold a pointer
float32 / float64             IEEE-754; float64 is the default
complex64 / complex128        complex numbers (rare in backend code)
byte                          alias for uint8 — a byte of data
rune                          alias for int32 — one Unicode code point
math.MaxInt64 / MinInt64  [*] limits live in the math package
math.MaxUint8 ...         [*] one constant per width
```

OTHER BASIC TYPES

```
bool                          true / false — no truthiness, no coercion
string                        immutable, UTF-8 encoded bytes
error                         the built-in interface { Error() string }
any                       [*] alias for interface{} (Go 1.18+)
nil                           zero value of pointer/slice/map/chan/func/interface
```

ZERO VALUES (no such thing as uninitialized)

```
numeric                       0
bool                          false
string                        "" (empty, not nil)
pointer / slice / map         nil
channel / func / interface    nil
struct                        every field at its own zero value
array                         every element at its own zero value
(a useful zero value is a design goal — see sync.Mutex, strings.Builder)
```

DECLARING & CONVERTING

```
var x int                     declare at the zero value
var x int = 5                 explicit type
var x = 5                     inferred
x := 5                        short form, function scope only
T(v)                          explicit conversion — Go never converts implicitly
float64(i)                    int -> float64
int(f)                        float64 -> int (truncates toward zero)
string(r)                     rune -> its UTF-8 encoding
string(65)                    "A" — a code point, NOT "65" (go vet warns)
strconv.Itoa(65)              "65" — the number as text
[]byte(s) / []rune(s)         string -> bytes / code points
untyped constant              adapts to context; no conversion needed
```

CONSTANTS & iota

```
const Pi = 3.14159           untyped: arbitrary precision until used
const MaxUsers int = 100     typed: strict from the start
const big = 1 << 30          evaluated at compile time
const ( A = iota; B; C )     0, 1, 2 — iota counts lines in the block
const ( _ = iota; KB = 1 << (10 * iota) )   skip 0, then 1024, 1048576...
type Weekday int             the idiomatic enum: a named type ...
const ( Sun Weekday = iota; Mon; Tue )      ... plus an iota block
func (d Weekday) String() string        [*] give the enum a name
(constants can only be basic types — no slices, maps, or structs)
```

NAMED TYPES vs ALIASES

```
type MyInt int               a NEW type: needs conversion, can have methods
type Byte = uint8            an ALIAS: the same type, interchangeable
type Celsius float64         named types make units type-safe
c := Celsius(21.5)           conversion required — that is the point
(you cannot add methods to a type from another package; wrap it instead)
```

OPERATORS

```
+ - * / %                    % is integer-only; / on ints truncates
== != < <= > >=              comparison -> bool
&& || !                      logical, short-circuiting
& | ^ &^ << >>               bitwise; &^ is AND NOT (bit clear)
+= -= *= /= %= &= |= ^=      compound assignment
++ --                        statements, not expressions: x++ only
&x / *p                      address-of / dereference
(no ternary operator — use a short if/else)
```

if

```
if x > 10 { ... }            no parentheses; braces always required
if err != nil { return err } the most common line in Go
if v, err := f(); err != nil scoped statement: v and err live in the if/else
} else if x > 5 {            else must sit on the closing brace's line
(the happy path stays at the left margin — return early on errors)
```

for — the only loop

```
for i := 0; i < n; i++ {}    the C-style three-clause form
for x < 10 {}                the "while" form
for {}                       the infinite form; exit with break/return
for i := range 10        [*] Go 1.22+: iterate 0..9 over an int
break / continue             leave / skip an iteration
break Outer / continue Outer  target a label on an outer loop
Outer: for ... {}            label a loop
goto Done                [*] legal, jumps to a label in the same function
(there is no do-while: use for { ...; if done { break } })
```

range

```
for i, v := range slice      index, copy of the element
for i := range slice         index only
for range slice          [*] neither — just repeat len(slice) times
for k, v := range m          map: RANDOM order, deliberately
for i, r := range s          string: byte index + rune (decodes UTF-8)
for v := range ch            channel: until closed and drained
for i := range n         [*] int: 0 .. n-1 (Go 1.22+)
for v := range seqFunc   [*] iterator function, iter.Seq (Go 1.23+)
(the loop variable is per-iteration since Go 1.22 — the old capture bug is gone)
```

switch

```
switch v { case 1: ... }     no fallthrough by default, no break needed
case 1, 2, 3:                several values in one case
switch { case x > 10: ... }  no subject: the cases are boolean expressions
switch v := f(); v {         scoped statement, like if
fallthrough                  explicitly run the next case body
default:                     the catch-all; position doesn't matter
switch x.(type) { ... }      type switch (see the interfaces sheet)
break                        leaves the switch, not the enclosing loop
```

defer

```
defer f()                    run f when the surrounding FUNCTION returns
defer mu.Unlock()            the pairing idiom: acquire, then defer release
defer file.Close()           args are evaluated NOW, the call happens later
defer func(){ ... }()        wrap in a closure to see final variable values
                             LIFO: the last defer registered runs first
defer func(){ recover() }()  the only place recover works
(deferred calls run on panic too — that's what makes them reliable)
```

TRAPS & MEMORIZE

```
int / int is integer div     3/2 == 1; convert first for a float result
string(intValue)             a code point, not the digits — use strconv
% on floats                  compile error; use math.Mod
map iteration order          randomized on purpose; sort keys for output
switch has no implicit fallthrough   the C habit is backwards here
defer in a loop              piles up until the function returns, not the iteration
defer's args evaluate early  defer log(x) captures x now
shadowing with :=            if err := ...; declares a NEW err inside the block
unused variable              compile error — assign to _ if truly unused
++ is a statement            x = y++ does not compile
comparing floats with ==     use an epsilon
untyped const overflow       const c = 1<<40; var x int8 = c fails at compile time
```